

“深入多线程内部机理”之二



Java内存模型

北京理工大学计算机学院
金旭亮

概述



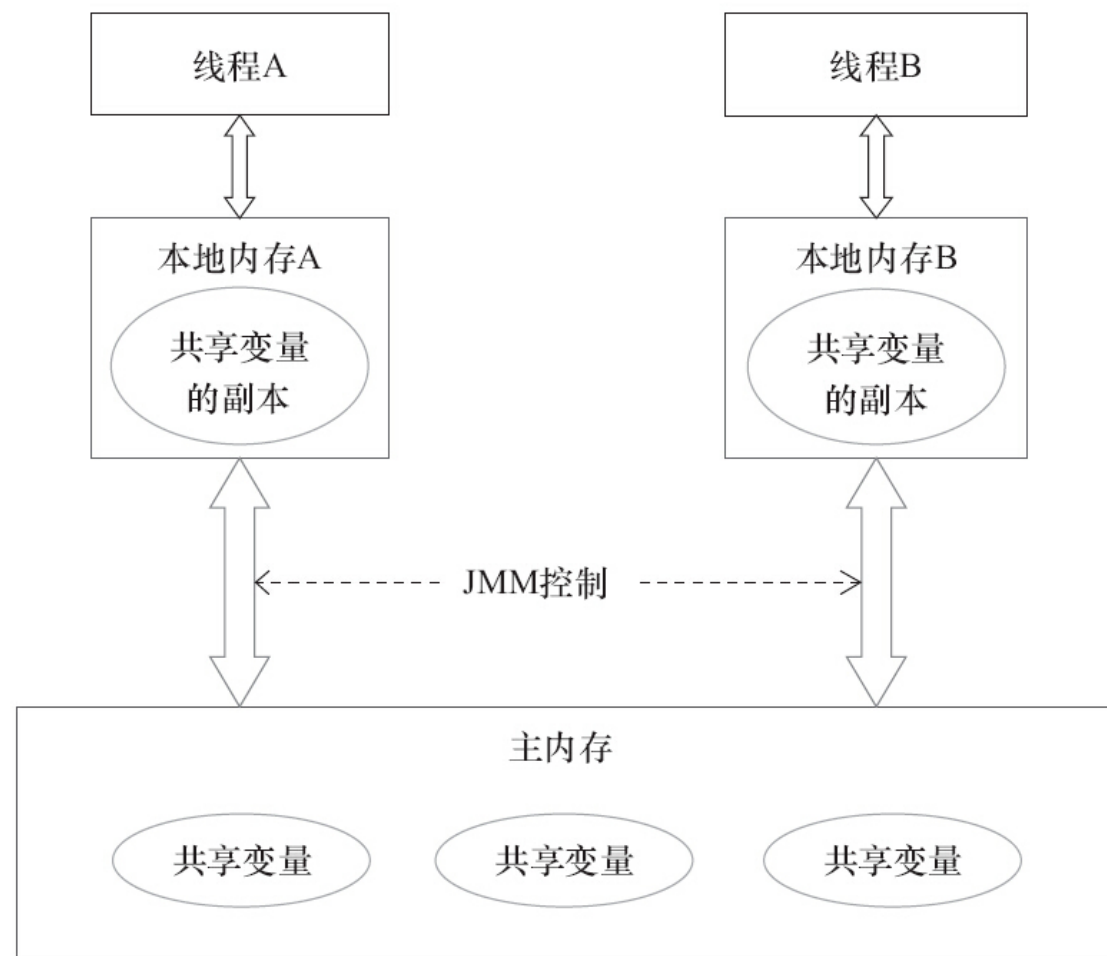
Java从设计之初，就考虑到了必须要“跨平台”，因此，必须要屏蔽掉底层特定计算机硬件与操作系统的之间的差异，提供一个统一的内存模型，以方便和统一各平台的Java虚拟机开发。

比如C语言中的整型变量，在一些平台下占用了两个字节的内存，而在另外些平台下则占用了四个字节的内存，为此，Java规定在任何平台下，Int类型统一定义为四个字节，这就是Java内存模型的主要功用——统一和规范。

Java内存模型 (JMM)

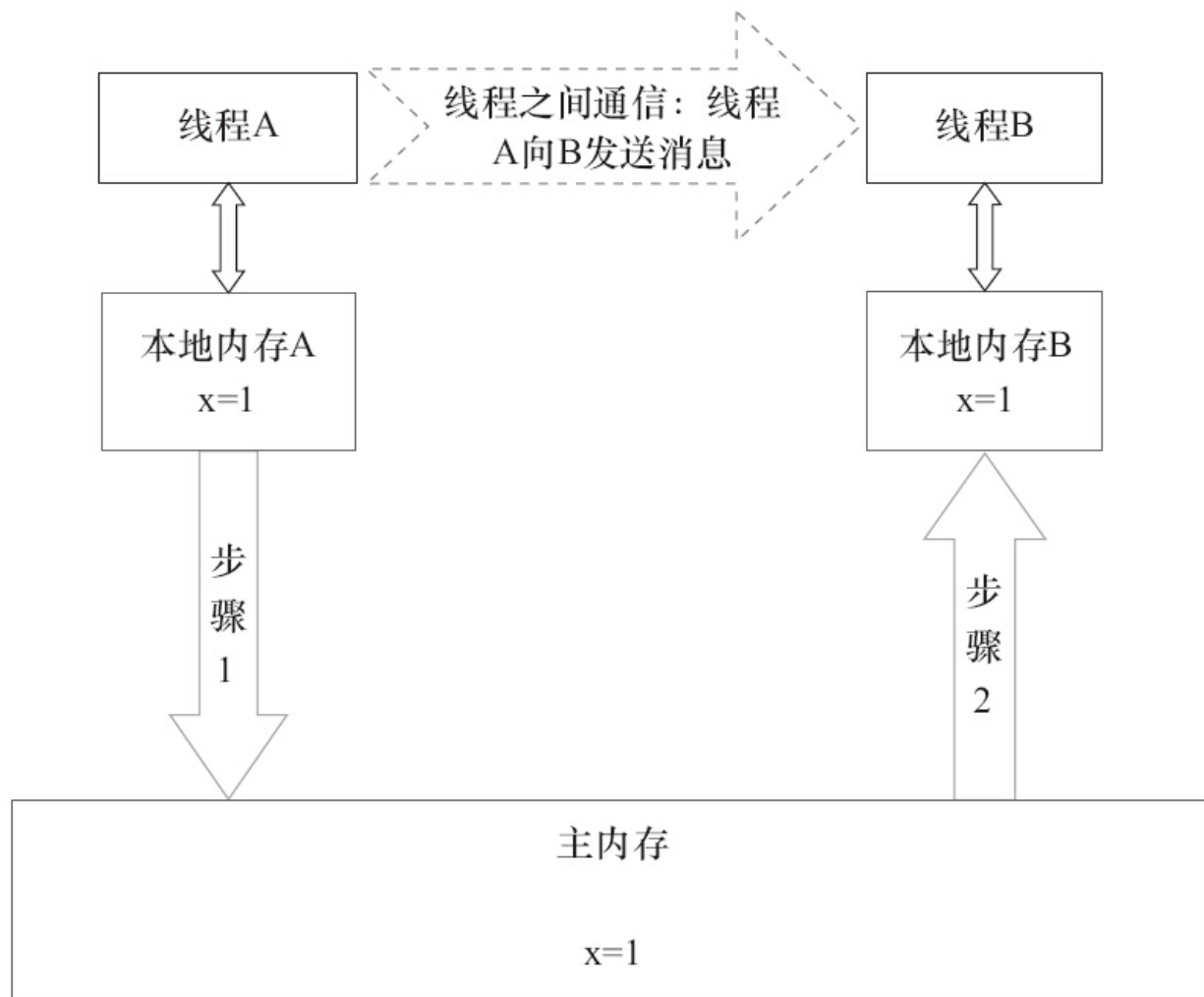
Java的内存模型规定了所有的变量都存在于主内存 (RAM)，而每个线程都有自己的工作内存或者本地内存，线程对变量的所有操作都必须在自己的工作内存中进行，而不能直接对主内存进行操作，并且每一个线程都不能访问其他线程的工作内存或者本地内存。

本地内存是JMM的一个抽象概念，并不真实存在。



本地内存与主内存之间的数据同步，由Java虚拟机的开发者负责，Java应用程序开发者不需要理会。

基于Java内存模型，实现线程之间的通信

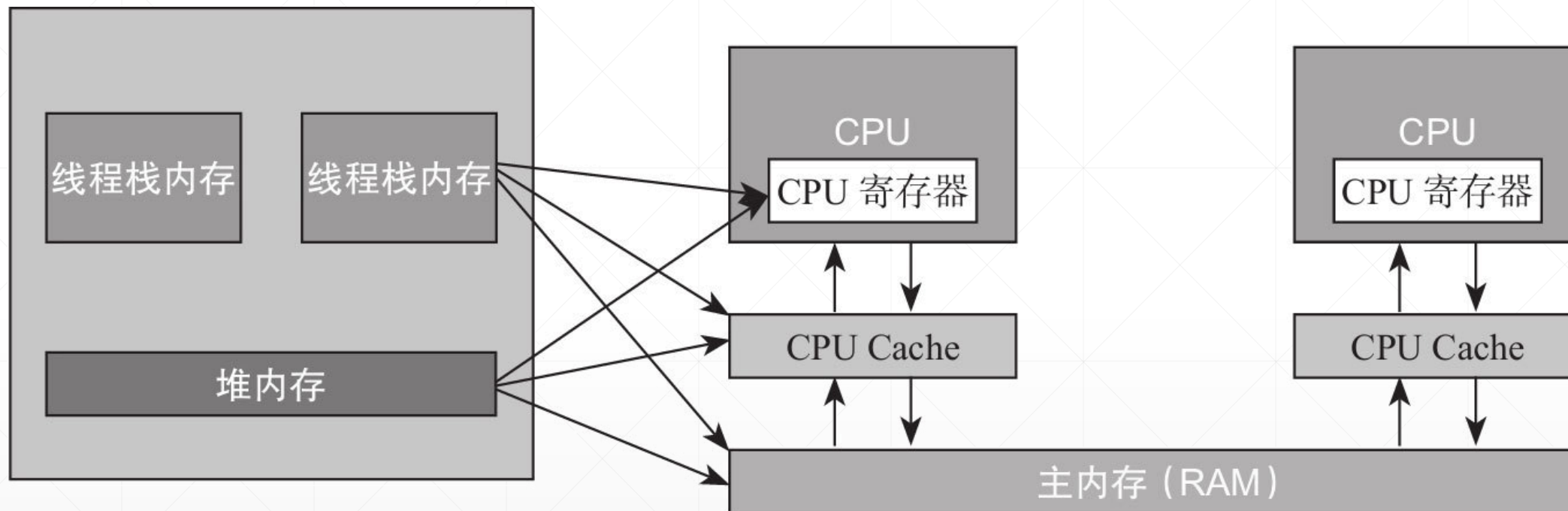


如果线程A与线程B之间要通信的话，必须要经历下面2个步骤。

- 1) 线程A把本地内存A中更新过的共享变量刷新到主内存中去。
- 2) 线程B到主内存中去读取线程A之前已更新过的共享变量。

这两个步骤实质上是线程A在向线程B发送消息，而且这个通信过程必须要经过主内存。JMM通过控制主内存与每个线程的本地内存之间的交互，来为Java程序员提供内存**可见性**保证。

Java内存模型与CPU硬件架构交互图



Java线程所使用的“工作内存”，是一个抽象的概念，在“真实的JVM中”，实现它涉及到了CPU缓存、寄存器、编译器优化以及物理内存之间的协同配合。

小结：Java内存模型要点



共享变量存储于主内存之中，每个线程都可以访问。



每个线程关联着一个私有的工作内存，称为“本地内存”。



对于需要在多线程间共享的变量，线程的工作内存保存的是它的副本。



线程不能直接操作主内存，必须先将数据值先写入自己的工作内存，然后再由Java内存模型的具体实现（即JVM）将数据值写入主内存中，然后其他线程才能读到新值。